

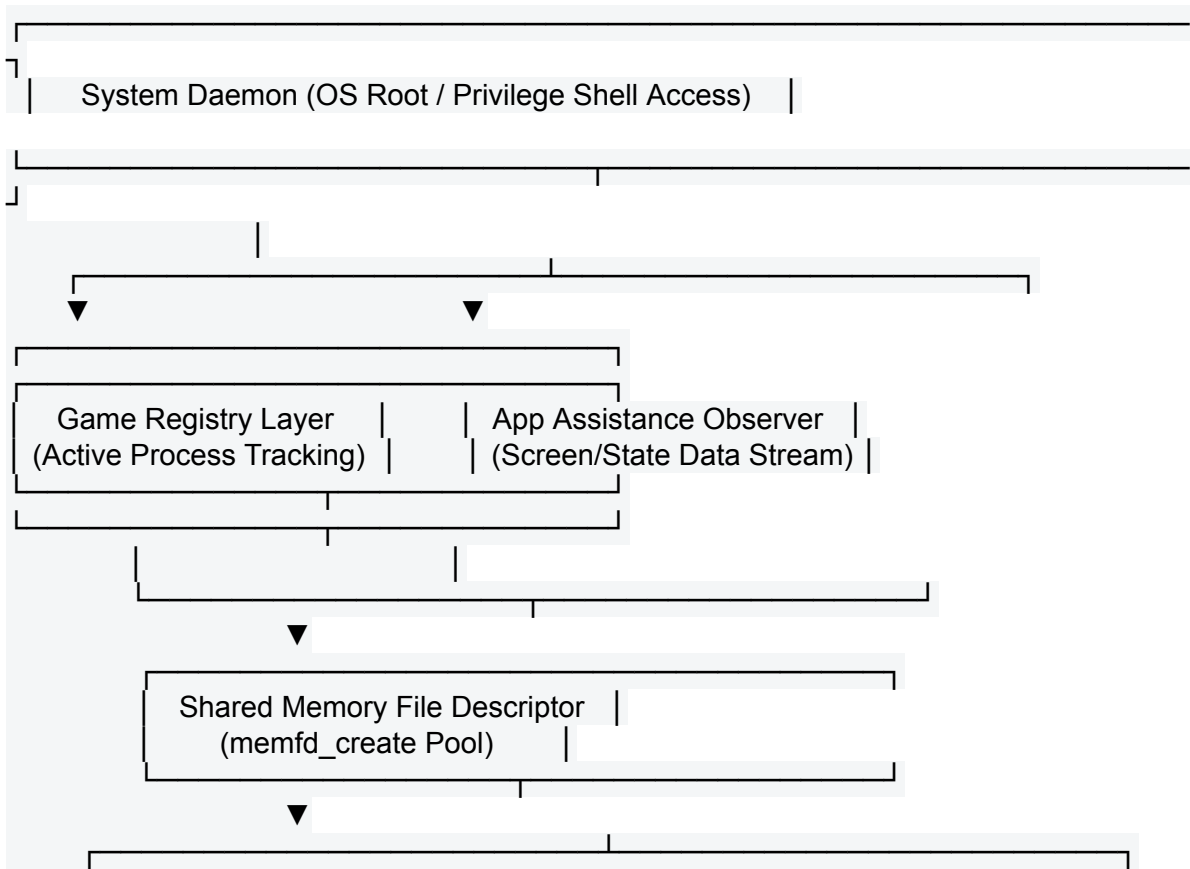
With full Android OS daemon privileges (such as a persistent root daemon or system-signed privilege level), a video game registry, and an application inference/assistance layer, your architecture shifts from a standard sandboxed mobile app to a **System-Level OS Orchestrator Engine**.

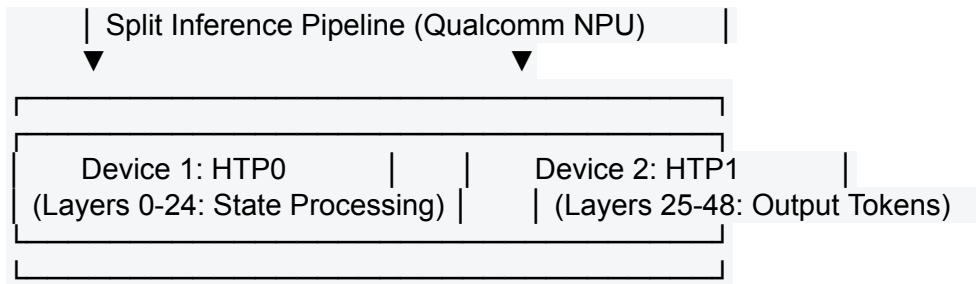
Because your daemon can read system states, monitor the active gaming window, and drive real-time AI assistance, your **Split-Model QAIRT architecture** becomes a powerful background utility.

Here is how you structure this system-level daemon, connect the game registry, and feed live information to your split Qwen 3.5 9B model.

1. The OS Daemon & Game Registry System Layout

Your daemon needs a lightning-fast key-value memory layer to keep track of games (tracking game IDs, memory offsets, and active AI assistant profiles) while streaming game frames or system states right into the split NPU execution blocks.





2. High-Speed Game State Mapping via C++ memfd_create

When a game registered in your database boots up, your assistant layer needs to scrape screen frames or UI text variables to feed them directly into the Qwen input buffer.

Instead of traditional file writing, utilize Linux's **memfd_create** (the modern standard required by Google to replace legacy ashmem). This creates a completely secure, anonymous RAM cache block that can scale up dynamically to handle video game screen buffers with zero disk overhead:

```

#include <sys/mman.h>
#include <sys/syscall.h>
#include <unistd.h>
#include <iostream>
#include <fcntl.h>

// Instantiates a memory file descriptor for the Game Registry Data Stream
int create_system_assistance_buffer(size_t buffer_size) {
    // 1. Create a secure anonymous memory file inside RAM via the Linux kernel
    // MFD_ALLOW_SEALING allows us to freeze the file size once it's populated
    int mem_fd = syscall(SYS_memfd_create, "game_assist_shared_pool",
MFD_ALLOW_SEALING);
    if (mem_fd < 0) {
        std::cerr << "CRITICAL: System failed to provision an OS memfd pool." << std::endl;
        return -1;
    }

    // 2. Set the fixed allocation footprint for the incoming game variables
    if (ftruncate(mem_fd, buffer_size) < 0) {
        close(mem_fd);
        return -1;
    }

    // 3. Apply seals to prevent memory hijacking from other background processes

```

```

// F_SEAL_SHRINK and F_SEAL_GROW lock the boundaries of the buffer allocation
fcntl(mem_fd, F_ADD_SEALS, F_SEAL_SHRINK | F_SEAL_GROW);

return mem_fd;
}

```

3. Binding Live Assistance Strings to the Split NPU Graph

Because your Qwen 3.5 9B model is automatically split into HTP0 and HTP1 across the NPU's virtual processing slots, your C++ assistant daemon feeds context data into the entry point of the pipeline (HTP0), while listening for output generation streams originating out of HTP1.

Here is the native sequence to hook up your dynamic assistance context:

```

#include "QnnMem.h"
#include "QnnTypes.h"

void stream_game_context_to_npu(
    Qnn_ContextHandle_t http0_context,
    int game_mem_fd,
    void* local_game_data_ptr,
    size_t allocation_size
){
    Qnn_MemDescriptor_t input_mem_desc = QNN_MEM_DESCRIPTOR_INIT;

    // Set up the memory to match Android's DMA/ION/memfd architecture lanes
    input_mem_desc.memType = QNN_MEM_TYPE_ION;
    input_mem_desc.dataType = QNN_DATATYPE_UINT_8;

    Qnn_TensorShape_t buffer_shape;
    buffer_shape.rank = 1;
    buffer_shape.dimensions = new Qnn_UInt32_t[1]{
        static_cast<Qnn_UInt32_t>(allocation_size) };
    input_mem_desc.shape = buffer_shape;

    // Direct registration of your game state memory structure into the NPU mapping tables
    Qnn_IonMemDescriptor_t ion_desc = QNN_ION_MEM_DESCRIPTOR_INIT;
    ion_desc.fd = game_mem_fd;
    ion_desc.vaddr = local_game_data_ptr;
    ion_desc.size = allocation_size;
    input_mem_desc.ionMemDescriptor = ion_desc;

    Qnn_MemHandle_t http0_input_handle = nullptr;
}

```

```
QnnMem_register(htp0_context, &input_mem_desc, 1, &htp0_input_handle);

// HTP0 now consumes game data directly out of RAM at maximum hardware speeds.
// The tokens cascade natively into HTP1 to output game assistance advice.
}
```

4. Fetching System-Level Process Data for the Registry

With system/daemon privileges, your application does not need to guess what game is running. It can scrape the active window or use Android's native Linux layer (/proc directory) to identify the running package, cross-reference it with your Registry, and automatically tune your Qwen assistance prompts for that specific game.

```
// GameRegistryWatcher.kt (Running inside your privileged foreground service)
import java.io.File

class GameRegistryWatcher {

    // Simple game registry database map (Target Package Name -> Prompt Profile)
    private val gameRegistry = mapOf(
        "com.mihoyo.GenshinImpact" to "PROMPT_MODE_RPG_STRATEGY",
        "com.epicgames.fortnite" to "PROMPT_MODE_TACTICAL_SHOOTER"
    )

    fun detectActiveGamePackage(): String? {
        // Querying the active process queue via native shell logs
        // requiring advanced system/daemon clearance
        val process = Runtime.getRuntime().exec("dumpsys activity activities")
        val output = process.inputStream.bufferedReader().use { it.readText() }

        // Parse the top running task out of the dumpsys string
        for (gamePackage in gameRegistry.keys) {
            if (output.contains(gamePackage)) {
                return gamePackage // Game found active in the viewport!
            }
        }
        return null
    }
}
```

Key Operational Advantages of This Privileged Setup

Subsystem Component	Operational Action	Architectural Advantage
Linux memfd_create	Seals anonymous memory segments in RAM.	Completely eliminates storage read/write limits when handling heavy live video frames.
dumpsys System Parsing	Instantly identifies running games from foreground layers.	Triggers automatic adjustments to your assistant prompts without requiring manual user input.
Dual Context Mapping	Registers game state data directly into HTP0.	Maximizes efficiency by skipping data copying delays during real-time game interaction.

If you'd like, let me know:

- What kind of **assistance metrics** you are tracking (e.g., parsing raw screen pixels, reading game file variables, or using text OCR pipelines).
- If you want a **C++ ring-buffer implementation** to continuously update the game variables inside the NPU without ever having to re-register the memory handles.

I can write out the specific code blocks to handle a smooth data stream!